

Instituição

Disciplina

Professor

Guia de Estudo: Implementação do Parser Dinâmico no GenESyS

Arquitetura, Bison/Flex, C++ dinâmico e integração com o simulador

Autor: Nome do autor

Matrícula: Matrícula

30 de junho de 2026

Sumário

1	Como Usar Este Guia	1
2	Visão do GenESyS que Precisamos Entender	1
2.1	O Model como centro da simulação	1
2.2	ModelDataDefinition, ModelComponent e ModelDataManager	1
3	Como a Linguagem de Expressões Funcionava Antes	2
3.1	Parser ativo por interface	2
3.2	O parser fixo e os arquivos-base	2
4	O Problema Arquitetural	3
5	Bison, Flex e C++ Dinâmico Neste Projeto	3
5.1	Bison e Flex no fluxo do GenESyS	3
5.2	Biblioteca compartilhada e carregamento em runtime	4
6	Os Contratos que Já Existiam	4
6.1	ParserChangesInformation	4
6.2	_getParserChangesInformation()	4
6.3	ParserManager	5
7	Implementação Passo a Passo	5
7.1	Passo 1: inserir uma extensão no modelo	5
7.2	Passo 2: o primeiro uso dispara o lazy reload	5
7.3	Passo 3: agregar mudanças de todos os objetos	6
7.4	Passo 4: gerar os arquivos Bison/Flex modificados	6
7.5	Passo 5: executar Bison, Flex e compilador C++	6
7.6	Passo 6: conectar o parser novo	7
7.7	Passo 7: trocar o parser sem perder o sampler	7
8	Exemplo Didático: ParserExtensionTestPlugin	7
9	Decisões de Engenharia	8
10	Diagramas Didáticos	9
10.1	Antes: parser fixo	9
10.2	Depois: atualização sob demanda	9
10.3	Pipeline de geração	9
11	Como Ler o Código	9
12	FAQ	10
13	Glossário Direcionado	11
14	Fechamento	11

1 Como Usar Este Guia

Este documento foi escrito para estudantes de Ciência da Computação que já conhecem a ideia de lexer, parser, token e gramática, mas ainda não conhecem bem o GenESyS, Bison, Flex ou o uso de bibliotecas dinâmicas em C++. O objetivo não é repetir o relatório técnico formal. A intenção é construir o caminho mental necessário para ler o código e entender por que as mudanças foram feitas.

O melhor modo de leitura é sequencial. Primeiro, o guia apresenta o papel do parser dentro do simulador. Depois, mostra como a linguagem de expressões era formada a partir de arquivos Bison/Flex. Em seguida, introduz os contratos já existentes no GenESyS e, finalmente, explica o mecanismo implementado passo a passo.

2 Visão do GenESyS que Precisamos Entender

2.1 O Model como centro da simulação

No GenESyS, a classe `Model` é o objeto central de uma simulação. Ela representa o modelo discreto que será executado: guarda componentes, dados auxiliares, eventos futuros, serviços de persistência, verificador de modelo, parser e gerenciadores internos.

Para entender o trabalho do parser dinâmico, a parte mais importante é perceber que cada `Model` nasce com três peças relacionadas à linguagem de expressões:

- um parser ativo, acessado pela interface `Parser\_if`;
- um sampler, usado por funções estocásticas da linguagem;
- um `ParserManager`, responsável por gerar e conectar parsers dinâmicos.

No construtor do `Model`, o parser e o gerenciador são criados como parte normal da infraestrutura do modelo:

```
_parser = new TraitsKernel<Parser_if>::Implementation(  
    this,  
    new TraitsKernel<Sampler_if>::Implementation());  
  
_parserManager = new ParserManager();  
_parserManager->setModel(this);  
_parserManager->setSourceDir(sourceRoot.string());  
_parserManager->setWorkDir(  
    (std::filesystem::temp_directory_path() /  
    "genesys_parser_auto").string());
```

Este trecho tem duas consequências importantes. Primeiro, o parser dinâmico não é um objeto separado do simulador; ele pertence ao ciclo de vida do `Model`. Segundo, o `ParserManager` já sabe qual modelo deve atualizar e onde procurar os arquivos-fonte do parser.

2.2 ModelDataDefinition, ModelComponent e ModelDataManager

O GenESyS organiza os elementos de modelagem em duas categorias importantes:

- **ModelComponent**: componentes do fluxo de simulação, como blocos que recebem entidades e executam ações;

- **ModelDataDefinition**: dados do modelo, como filas, recursos, variáveis, conjuntos, fórmulas e objetos auxiliares.

Na prática, **ModelComponent** herda de **ModelDataDefinition**. Por isso, o projeto usa **ModelDataDefinition** como ponto comum para perguntar: “este objeto precisa mudar a linguagem do parser?”.

O **ModelDataManager** funciona como um catálogo. Ele guarda objetos por tipo e nome, por exemplo todas as filas, todos os recursos, todas as variáveis. O lexer do GenESyS usa esse catálogo para reconhecer nomes de objetos dentro de expressões. Quando o usuário escreve algo como uma referência a uma fila, o scanner pode consultar o **ModelDataManager** para descobrir se aquele texto corresponde a uma definição existente.

3 Como a Linguagem de Expressões Funcionava Antes

3.1 Parser ativo por interface

O restante do simulador não chama diretamente uma classe concreta do parser. Ele chama a interface **Parser_if**. Essa interface define operações como avaliar uma expressão, consultar erro, associar sampler e expor o driver interno quando necessário.

```
class Parser_if {
public:
    virtual double parse(const std::string expression) = 0;
    virtual double parse(const std::string expression,
                        bool& success,
                        std::string& errorMessage) = 0;
    virtual void setSampler(Sampler_if* sampler) = 0;
    virtual Sampler_if* getSampler() const = 0;
    virtual genesyspp_driver getParser() const = 0;
};
```

Esse desenho é importante porque permite trocar a implementação concreta do parser sem mudar todas as classes que avaliam expressões. Antes do parser dinâmico, essa interface normalmente apontava para **ParserDefaultImpl2**, que encapsula o driver gerado por Bison/Flex.

3.2 O parser fixo e os arquivos-base

A linguagem de expressões do GenESyS vem de dois arquivos principais:

- **source/parser/parserBisonFlex/lexerparser.ll**, usado pelo Flex para gerar o scanner;
- **source/parser/parserBisonFlex/bisonparser.yy**, usado pelo Bison para gerar o parser.

Esses arquivos já possuíam regiões marcadas para trechos de plugins. Por exemplo, o arquivo Bison tem uma seção de tokens de plugins:

```
/*****begin_Tokens_plugins*****/
%token <obj_t> QUEUE
%token <obj_t> fNQ
%token <obj_t> fFIRSTINQ
/*****end_Tokens_plugins*****/
```

O arquivo Flex também tem uma região para regras léxicas de plugins:

```
%{ /****begin_Lexical_plugins****/%}  
[nN] [qQ] {return yy::genesyspp_parser::make_fNQ(  
    obj_t(0, std::string(ytext)), loc);} ;  
%{ /****end_Lexical_plugins****/%}
```

Esses marcadores mostram que a arquitetura já imaginava extensões de linguagem. O que faltava era um mecanismo concreto para pegar fragmentos declarados por plugins, injetar nesses pontos e recompilar um parser novo automaticamente.

4 O Problema Arquitetural

Um plugin pode adicionar uma classe nova ao simulador. Por exemplo, uma nova definição de dados ou um novo componente. Mas isso não significa que a linguagem de expressões passe a reconhecer automaticamente novas funções, nomes ou tokens.

Antes desta implementação, havia um desalinhamento:

- o modelo podia receber novas classes;
- os arquivos Bison/Flex tinham marcadores para extensões;
- mas não havia pipeline completo para regenerar e conectar o parser em runtime.

No código real, vários plugins já sobrescrevem `_getParserChangesInformation()`, mas retornam um objeto vazio. Este é um sinal de que o contrato existia, mas muitas contribuições ainda não estavam implementadas. Um exemplo simplificado encontrado em plugins reais é:

```
ParserChangesInformation* Queue::_getParserChangesInformation() {  
    ParserChangesInformation* changes = new ParserChangesInformation();  
    // changes->getProductionToAdd()->insert(...);  
    // changes->getTokensToAdd()->insert(...);  
    return changes;  
}
```

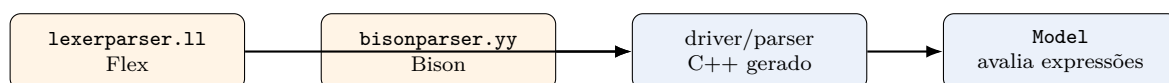
Por isso, os testes de unidade usam uma classe chamada `ParserExtensionTestPlugin`. Ela é o melhor exemplo didático porque declara uma extensão completa: token Bison, produção Bison e regra Flex.

5 Bison, Flex e C++ Dinâmico Neste Projeto

5.1 Bison e Flex no fluxo do GenESyS

Como vocês já conhecem parser e lexer, o ponto importante aqui não é explicar o conceito, mas a ferramenta. Flex recebe um arquivo `.ll` e gera código C++ para o scanner. Bison recebe um arquivo `.yy` e gera código C++ para o parser.

No GenESyS, o pipeline estático tradicional pode ser pensado assim:



A implementação dinâmica não muda a teoria do parser. Ela automatiza a geração de novos arquivos `.ll/.yy`, executa Flex/Bison e compila uma nova biblioteca compartilhada.

5.2 Biblioteca compartilhada e carregamento em runtime

Em Linux, uma biblioteca compartilhada normalmente usa extensão `.so`. Ela pode ser carregada durante a execução do programa com `dlopen`. Depois de carregada, o programa pode procurar uma função dentro dela com `dlsym`.

O problema em C++ é que nomes de funções podem sofrer *name mangling*. Ou seja, o nome real exportado no binário pode não ser exatamente o texto escrito no código. Por isso, o projeto usa uma *factory C*:

```
extern "C" Parser_if* genesys_createParser(Model* model,
                                          Sampler_if* sampler) {
    return new ParserDefaultImpl2(model, sampler);
}
```

O `extern "C"` preserva um nome estável para a função. Assim, o kernel pode procurar exatamente `genesys\_createParser` com `dlsym`, sem depender de detalhes da ABI C++.

6 Os Contratos que Já Existiam

6.1 ParserChangesInformation

`ParserChangesInformation` é um objeto de transporte. Ele não compila nada, não carrega biblioteca e não avalia expressões. Ele apenas carrega fragmentos de texto que serão inseridos nos arquivos Bison/Flex.

```
class ParserChangesInformation {
public:
    std::string gettokens() const;
    void setTokens(const std::string& newTokens);

    std::string getfunctionProductions() const;
    void setFunctionProductions(const std::string& newFunctionProductions);

    std::string getlexicalRules() const;
    void setLexicalRules(const std::string& newLexicalRules);

    bool hasChanges() const;
};
```

Pense nesse objeto como um “pacote de patches” para o parser. Cada campo corresponde a uma região do arquivo Bison ou Flex.

6.2 _getParserChangesInformation()

`ModelDataDefinition` define um hook virtual:

```
ParserChangesInformation*
ModelDataDefinition::_getParserChangesInformation() {
    return nullptr;
}
```

O retorno `nullptr` significa: “esta classe não participa da atualização dinâmica do parser”. Uma subclasse pode sobrescrever esse método e retornar um objeto com tokens, produções ou regras léxicas.

6.3 ParserManager

ParserManager é o coordenador. Ele não decide quais mudanças uma extensão quer fazer, mas sabe como coletar, gerar, compilar e conectar um parser novo.

```
struct GenerateNewParserResult {
    bool result = false;
    std::string bisonMessages;
    std::string lexMessages;
    std::string compilationMessages;
    NewParser newParser;
};
```

Esse resultado é didaticamente importante: o pipeline tem várias etapas que podem falhar. Separar mensagens de Bison, Flex e compilação ajuda a entender onde o problema ocorreu.

7 Implementação Passo a Passo

7.1 Passo 1: inserir uma extensão no modelo

Quando um objeto entra no modelo, ele passa pelo `ModelDataManager::insert()`. Depois de registrar o objeto, o gerenciador pergunta se esse objeto contribui com mudanças de parser.

```
ParserChangesInformation* changes =
    anElement->_getParserChangesInformation();

if (changes != nullptr && changes->hasChanges()) {
    _parentModel->setParserIsStale(true);
}

delete changes;
```

O ponto central é que `insert()` não recompila o parser. Ele apenas marca o parser como possivelmente desatualizado. Esse estado é armazenado em `Model::_parserIsStale`.

7.2 Passo 2: o primeiro uso dispara o lazy reload

O reload acontece em `Model::parseExpression(expression, success, errorMessage)`. Esse método é uma boa escolha porque é o ponto em que a linguagem realmente precisa estar atualizada.

```
if (_parserIsStale) {
    _parserIsStale = false;
    auto allChanges = _parserManager->aggregateChanges();
    auto* combined = new ParserChangesInformation();
    for (auto* ch : allChanges) {
        combined->setTokens(
            combined->gettokens() + ch->gettokens());
        combined->setLexicalRules(
            combined->getlexicalRules() + ch->getlexicalRules());
    }
    auto result = _parserManager->generateNewParser(combined);
    ...
}
```

O trecho acima está reduzido: no código real, todos os campos são combinados. A ideia é simples: se existem várias extensões, suas contribuições são concatenadas por seção antes da geração.

7.3 Passo 3: agregar mudanças de todos os objetos

`ParserManager::aggregateChanges()` percorre todos os tipos de `ModelDataDefinition` registrados no `ModelDataManager`. Para cada objeto, chama o hook virtual.

```
ParserChangesInformation* changes =
    mdd->_getParserChangesInformation();

if (changes != nullptr && changes->hasChanges()) {
    result.push_back(changes);
} else {
    delete changes;
}
```

Esse método implementa a ideia de contrato. O `ParserManager` não precisa saber se o objeto é fila, recurso, variável ou plugin de teste. Ele só sabe chamar o método virtual e aceitar objetos que realmente tenham mudanças.

7.4 Passo 4: gerar os arquivos Bison/Flex modificados

`generateNewParser()` começa localizando os arquivos-base e lendo seu conteúdo. Depois, injeta os fragmentos nos marcadores.

```
bool ParserManager::_injectSection(std::string& content,
    const std::string& beginMarker,
    const std::string& endMarker,
    const std::string& block) {
    if (block.empty()) return true;
    size_t beginPos = content.find(beginMarker);
    if (beginPos == std::string::npos) return false;
    size_t endPos = content.find(endMarker, beginPos);
    if (endPos == std::string::npos) return false;
    content.insert(endPos, block + "\n");
    return true;
}
```

Essa função evita gerar uma gramática paralela. O projeto usa a gramática original como fonte de verdade e apenas adiciona fragmentos nos pontos planejados.

7.5 Passo 5: executar Bison, Flex e compilador C++

Depois da injeção, o gerenciador escreve novos arquivos no diretório de trabalho e executa as ferramentas:

```
bison --defines=... --locations --debug -Wall -v ...
flex -o Genesys++-scanner.cpp lexerparser.ll
g++ -std=c++23 -fPIC -shared ... -o genesys_parser_dynamic.so
```

O resultado é uma biblioteca compartilhada chamada `genesys\_parser\_dynamic.so`. Ela contém o parser gerado, o scanner gerado, arquivos de suporte e a factory `genesys\_createParser`.

7.6 Passo 6: conectar o parser novo

`connectNewParser()` valida se o arquivo existe, carrega a biblioteca e resolve a factory.

```
void* newLibraryHandle =
    dlopen(newParser.compiledParserFilename.c_str(),
           RTLD_NOW | RTLD_DEEPBIND);

auto* createFn =
    reinterpret_cast<Parser_if* (*)(Model*, Sampler_if*)>(
        dlsym(newLibraryHandle, "genesys_createParser"));
```

`RTLD_NOW` faz a resolução de símbolos acontecer imediatamente. `RTLD_DEEPBIND` ajuda a biblioteca carregada a preferir seus próprios símbolos, reduzindo conflito com símbolos já presentes no processo.

7.7 Passo 7: trocar o parser sem perder o sampler

O parser usa um `Sampler_if` para funções estocásticas. Ao trocar o parser, esse sampler precisa continuar vivo e não pode ser destruído duas vezes. Por isso, a interface ganhou operações explícitas de ownership.

```
Sampler_if* ParserDefaultImpl2::releaseSampler() {
    Sampler_if* s = _wrapper.getSampler();
    _ownsSampler = false;
    _wrapper.setSampler(nullptr);
    return s;
}
```

No `Model::setParser()`, o sampler sai do parser antigo e entra no novo:

```
Sampler_if* currentSampler =
    (_parser != nullptr) ? _parser->releaseSampler() : nullptr;
if (parser != nullptr && currentSampler != nullptr) {
    parser->setSamplerOwned(currentSampler);
}
delete _parser;
_parser = parser;
```

Sem essa transferência, a troca dinâmica poderia causar vazamento de memória, perda do sampler ou dupla liberação.

8 Exemplo Didático: ParserExtensionTestPlugin

O teste cria uma extensão que adiciona a função `parsertest(x)`. Ela retorna $x + 1$. Para isso, declara um token, uma produção Bison e uma regra Flex.

```
changes->setTokens("%token <obj_t> fPARSERTEST\n");

changes->setFunctionProductions(
    " | fPARSERTEST \"(\" expression \")\" \"
    \"{ $$.valor = $3.valor + 1.0; }\n");

changes->setLexicalRules(
    "[pP][aA][rR][sS][eE][rR][tT][eE][sS][tT] \"
    \"{ return yy::genesyspp_parser::make_fPARSERTEST(\"
    \"obj_t(0, std::string(yytext)), loc); }\n");
```

Esse exemplo é pequeno, mas mostra a ideia completa:

- Flex reconhece o texto `parsertest`;
- o scanner retorna o token `fPARSERTEST`;
- Bison aceita `fPARSERTEST "(" expression ")"`;
- a ação semântica calcula o valor final.

Depois que o parser novo é conectado, os testes validam expressões como:

```
model->parseExpression("parsertest(5)", success, errorMsg);  
model->parseExpression("parsertest(parsertest(2))");
```

9 Decisões de Engenharia

Decisão	Motivo	Onde aparece
Lazy reload	Evita recompilar a cada inserção e só paga o custo quando uma expressão precisa ser avaliada.	<code>ModelDataManager::insert()</code> e <code>Model::parseExpression()</code>
Injeção em marcadores	Mantém os arquivos-base como fonte de verdade e evita uma gramática paralela.	<code>ParserManager::_injectSection()</code>
Parser antigo preservado	Uma falha em Bison, Flex, compilação ou conexão não deve inutilizar o simulador.	<code>connectNewParser()</code> e retorno de erro em <code>parseExpression()</code>
Factory C	Garante nome exportado estável apesar do name mangling C++.	<code>ParserFactory.cpp</code>
Ownership explícito do sampler	Evita vazamento, dupla liberação e perda de estado estocástico.	<code>Parser_if</code> , <code>ParserDefaultImpl2</code> , <code>Model::setParser()</code>

10 Diagramas Didáticos

10.1 Antes: parser fixo

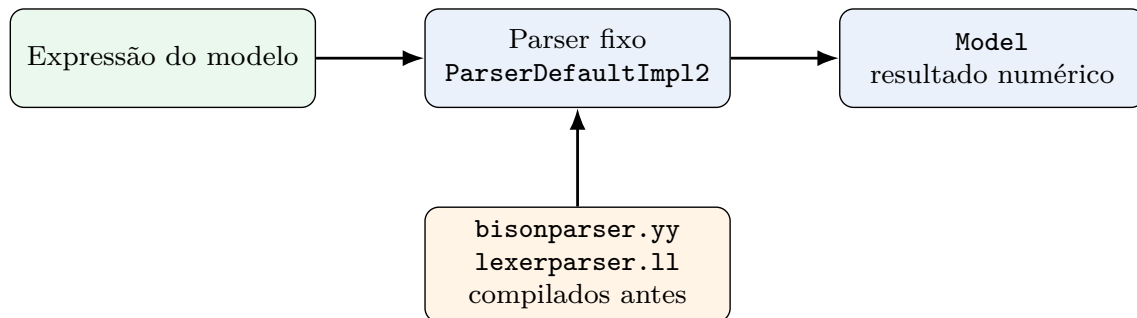


Figura 1: Antes da solução, o parser ativo refletia a linguagem já compilada.

10.2 Depois: atualização sob demanda



Figura 2: Depois da solução, a atualização acontece automaticamente no primeiro uso da linguagem.

10.3 Pipeline de geração



Figura 3: Pipeline completo entre uma contribuição de linguagem e o parser ativo.

11 Como Ler o Código

Arquivo ou classe	O que procurar	Pergunta respondida
ParserChangesInformation	Campos, getters/setters e <code>hasChanges()</code> .	Como uma extensão descreve mudanças?
ModelDataDefinition	Hook <code>_getParserChangesInformation()</code>	Como uma classe participa do contrato?
ModelDataManager::insert()	Marcação de <code>parserIsStale</code> .	Quando o modelo percebe que o parser pode estar velho?
Model::parseExpression()	Lazy reload antes de avaliar.	Quando a regeneração realmente acontece?

Arquivo ou classe	O que procurar	Pergunta respondida
<code>ParserManagerRuntime.cpp</code>	Agregação, <code>dlopen</code> , <code>dlsym</code> .	Como o parser novo entra no modelo?
<code>ParserManager.cpp</code>	Injeção, Bison, Flex, compilação.	Como a biblioteca <code>.so</code> é gerada?
<code>ParserFactory.cpp</code>	<code>extern "C"</code> e criação do parser.	Qual é a fronteira ABI estável?
<code>ParserDefaultImpl2</code>	Parser concreto e ownership do sampler.	Quem avalia expressões e quem destrói o sampler?
<code>test_parser_expressions</code>	<code>ParserExtensionTestPlugin</code>	Como uma extensão completa funciona na prática?

12 FAQ

Por que não recompilar no `insert()`?

Porque inserir objetos no modelo pode acontecer várias vezes em sequência. Se cada inserção recompilasse o parser, o custo seria alto e repetitivo. O lazy reload acumula mudanças e só recompila quando uma expressão precisa ser avaliada.

Por que os plugins reais ainda têm mudanças vazias?

Porque o contrato foi preparado para que plugins declarem mudanças, mas nem todo plugin já implementa uma contribuição real de linguagem. Por isso, o método `hasChanges()` é importante: ele evita tratar objeto vazio como alteração efetiva.

Por que os testes usam `ParserExtensionTestPlugin`?

Porque ele é pequeno e completo. Ele declara token, produção e regra léxica, permitindo validar o pipeline inteiro sem depender de um plugin grande do simulador.

O que acontece se Bison, Flex ou g++ falharem?

O resultado de geração retorna `result = false` e mensagens separadas. O `Model::parseExpression()` reporta erro e não substitui o parser ativo por um parser quebrado.

Por que usar `extern "C"`?

Para exportar uma função com nome estável. Sem isso, o compilador C++ poderia transformar o nome da função, e `dlsym` não encontraria `genesys\_createParser` pelo texto esperado.

Qual é o papel do sampler?

O sampler fornece suporte a funções estocásticas da linguagem. Quando o parser é trocado, esse objeto precisa sobreviver e passar para o parser novo. Por isso, a troca exige cuidado

com ownership.

13 Glossário Direcionado

Bison Ferramenta que gera parser C/C++ a partir de uma gramática em arquivo `.yy`.

Flex Ferramenta que gera scanner C/C++ a partir de regras léxicas em arquivo `.ll`.

.so Biblioteca compartilhada em Linux, carregável em runtime.

dlopen Função que abre uma biblioteca compartilhada durante a execução.

dlsym Função que procura um símbolo exportado dentro de uma biblioteca carregada.

ABI Contrato binário entre código compilado e chamadores.

Name mangling Transformação de nomes C++ feita pelo compilador para suportar overloads, namespaces e tipos.

Factory Função ou objeto que cria instâncias sem expor o construtor concreto ao chamador.

Hook virtual Método virtual usado como ponto de extensão por subclasses.

Lazy reload Atualização adiada até o primeiro momento em que ela é necessária.

Stale Estado em que o parser ativo ainda existe, mas pode estar desatualizado.

Ownership Regra de quem é responsável por destruir um objeto em C++.

Sampler Objeto usado pelo parser para suporte a funções estocásticas.

14 Fechamento

A implementação do parser dinâmico não muda apenas uma função isolada. Ela conecta contratos já presentes no GenESyS a um pipeline completo: extensões declaram fragmentos, o modelo marca o parser como desatualizado, o primeiro uso dispara a geração, Bison/Flex/g++ produzem uma biblioteca, e o kernel carrega essa biblioteca para substituir o parser ativo com segurança.

Para estudar o trabalho, vale lembrar a frase-resumo: o plugin não compila nada; ele declara. O **ParserManager** não inventa uma linguagem nova; ele injeta nos arquivos-base. O **Model** não pede intervenção manual; ele atualiza sob demanda quando a expressão precisa ser avaliada.